UNIVERSIDAD AUTONOMA DE NUEVO LEON

Facultad de Ciencias Físico Matemáticas

Algoritmia y optimización

Tarea 2

Nombre: Alejandro Rincón Guillen

Matricula: 2135395

Docente: José Anastacio Hernández Saldaña M.C.

Grupo: 031

1. Parallel Speedup: Run estimate_pi with 1, 2, 4 and 8 processes and plot the speedup. Why is it not linear?

Código base

Este es el código base.

```
import multiprocessing as mp
import random
import time
import matplotlib.pyplot as plt

def estimate_pi(seed: int, samples: int = 1_000_000) -> float:
    rng = random.Random(seed)
    inside = sum(rng.random()**2 + rng.random()**2 <= 1.0 for _ in range(samples))
    return 4 * inside / samples

if __name__ == '__main__':
    seeds = list(range(8))
```

Código modificado y ejecuciones

```
    # 1. Ejecución normal
    t0 = time.perf_counter()
    with mp.Pool(processes=mp.cpu_count()) as pool:
        pool.map(estimate_pi, seeds)
    print(f"Normal (Todos los núcleos) - Tiempo: {time.perf_counter() - t0:.3f} s")

    # 2. Ejecución modificada: 1, 2, 4 y 8 procesos
    procesos = [1, 2, 4, 8]
    tiempos = {}
    speedups = []

    print("\nEjecución Modificada:")
    for p in procesos:
        t0 = time.perf_counter()
        with mp.Pool(processes=p) as pool:
            pool.map(estimate_pi, seeds)
        t_par = time.perf_counter() - t0
        tiempos[p] = t_par

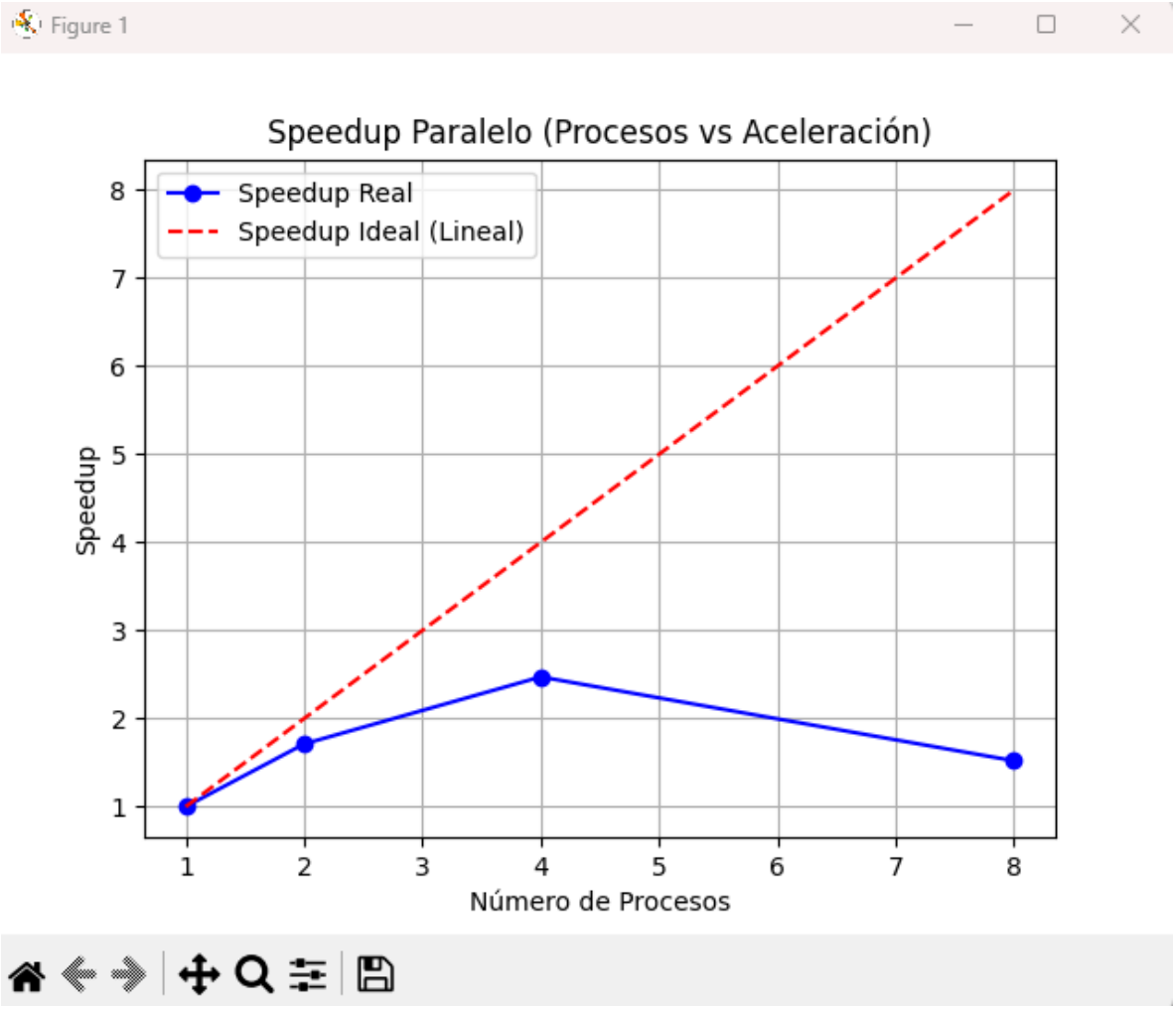
        speedup = tiempos[1] / t_par if p > 1 else 1.0
        speedups.append(speedup)
        print(f"Procesos: {p} | Tiempo: {t_par:.3f} s | Speedup: {speedup:.2f}x")
```

Resultado de la ejecución

```
Normal (Todos los núcleos) - Tiempo: 1.662 s

Ejecución Modificada:
Procesos: 1 | Tiempo: 3.831 s | Speedup: 1.00x
Procesos: 2 | Tiempo: 2.238 s | Speedup: 1.71x
Procesos: 4 | Tiempo: 1.549 s | Speedup: 2.47x
Procesos: 8 | Tiempo: 2.519 s | Speedup: 1.52x
```

Grafica de speedup



Explicación: ¿Por qué el Speedup no es lineal?

La aceleración incrementa al pasar de 1 a 4 procesos (alcanzando un máximo de 2.47x). Sin embargo, al usar 8 procesos el tiempo empeora y el speedup cae a 1.52x. Esto demuestra que no es lineal debido a:

1. **Límite de núcleos físicos:** Mi procesador solo tiene 4 núcleos reales. Lanzar 8 procesos provoca competencia y cambio de contexto.
2. **Sobrecarga de gestión:** El patrón Collector/Reducer (mp.Pool) requiere crear procesos y enviar datos a través de comunicación interproceso (IPC), añadiendo un tiempo extra que frena la ganancia en paralelo.

2. Algorithmic Analysis: Modify the greedy knapsack algorithm to prioritize items with the lowest weight first. Show a small instance where this performs worse than the value/weight ratio method.

Código base

Este es el código base.

```
def greedy_ratio(items, capacity):
    # Ordena por mayor ratio
    sorted_items = sorted(items, key=lambda x: x['v'] / x['w'], reverse=True)
    total_value = 0
    for item in sorted_items:
        if item['w'] <= capacity:
            total_value += item['v']
            capacity -= item['w']
    return total_value
```

Código modificado y ejecuciones

```
def greedy_lowest_weight(items, capacity):
    # Ordena priorizando el menor peso
    sorted_items = sorted(items, key=lambda x: x['w'])
    total_value = 0
    for item in sorted_items:
        if item['w'] <= capacity:
            total_value += item['v']
            capacity -= item['w']
    return total_value

# Prueba
items = [
    {'name': 'A', 'v': 10, 'w': 10},
    {'name': 'B', 'v': 20, 'w': 20},
    {'name': 'C', 'v': 1000, 'w': 40}
]
print(f"Mejor valor (Ratio v/w): {greedy_ratio(items, 50)}")
print(f"Mejor valor (Menor Peso): {greedy_lowest_weight(items, 50)}")
```

Resultado de la ejecución

```
Mejor valor (Ratio v/w): 1010
Mejor valor (Menor Peso): 30
```

Comparativa de Resultados:

- **Heurística de Menor Peso (Modificado):** Elige los objetos $\mathrm{A}\ (\mathrm{w} = 10, \mathrm{v} = 10)$ y $\mathrm{B}\ (\mathrm{w} = 20, \mathrm{v} = 20)$ por ser los más ligeros. Ocupa 30 de capacidad y obtiene un valor de 30. Descarta el objeto $\mathrm{C}\ (\mathrm{w} = 40, \mathrm{v} = 1000)$ porque ya no cabe.
- **Heurística de Ratio Valor/Peso (Original):** Elige el objeto $\mathrm{C}\ (\mathrm{w} = 40, \mathrm{v} = 1000)$ y luego el $\mathrm{A}\ (\mathrm{w} = 10, \mathrm{v} = 10)$. Ocupa la capacidad de 50 y obtiene un valor de 1010.
- **Solución Óptima Global (Programación Dinámica / Exacta):** Da un valor de 1010 con los objetos $\{\mathrm{C}, \mathrm{A}\}$.

Método	**Objetos Seleccionados**	**Capacidad Usada**	**Valor Total**	**Estado de Solución**
Menor Peso	$\{A, B\}$	30/50	30	Óptimo Local Pésimo
Ratio Valor/Peso	$\{C, A\}$	50/50	1010	Óptimo Global
Programación Dinámica	$\{C, A\}$	50/50	1010	Óptimo Global

Explicación:

Este ejemplo demuestra cómo los algoritmos greedy son "miopes": toman decisiones locales inmediatas sin ver el impacto final y no pueden corregir sus pasos. La regla de menor peso asume que meter muchos objetos livianos dará más valor. Sin embargo, aquí llena la mochila con "basura ligera", desperdicia el espacio y queda atrapada en un óptimo local pésimo de 30 frente al óptimo real de 1010.

3. Local Search: Implement 2-opt for the TSP and count how many improvements are made before reaching the local optimum. Does starting from a Nearest Neighbour tour require fewer improvements than a random tour?

Código base

Este es el código base.

```
import math
import random

def tour_length(route, cities):
    return sum(math.dist(cities[route[i]], cities[route[(i + 1) % len(route)]])
               for i in range(len(route)))

def two_opt_move(route, i, k):
    return route[:i] + route[i:k + 1][::-1] + route[k + 1:]
```

Código modificado y ejecuciones

```
def full_2opt_count(route, cities):
    improvements = 0
    improved = True
    while improved:
        improved = False
        for i in range(1, len(route) - 1):
            for k in range(i + 1, len(route)):
                new_route = two_opt_move(route, i, k)
                if tour_length(new_route, cities) < tour_length(route, cities):
                    route = new_route
                    improvements += 1
                    improved = True
                    break
            if improved:
                break
    return route, improvements

def nearest_neighbour(cities, start_node):
    unvisited = list(cities.keys())
    tour = [start_node]
    unvisited.remove(start_node)
    current = start_node

    while unvisited:
        next_node = min(unvisited, key=lambda city: math.dist(cities[current], cities[city]))
        tour.append(next_node)
        unvisited.remove(next_node)
        current = next_node

    return tour

#Prueba
if __name__ == '__main__':
    random.seed(42)
    cities = {f'C{i}': (random.uniform(0, 100), random.uniform(0, 100)) for i in range(15)}

    ruta_aleatoria = list(cities.keys())
    random.shuffle(ruta_aleatoria)
    dist_aleatoria_inicial = tour_length(ruta_aleatoria, cities)

    ruta_nn = nearest_neighbour(cities, 'C0')
    dist_nn_inicial = tour_length(ruta_nn, cities)

    ruta_aleatoria_final, mejoras_aleatoria = full_2opt_count(ruta_aleatoria, cities)
    ruta_nn_final, mejoras_nn = full_2opt_count(ruta_nn, cities)

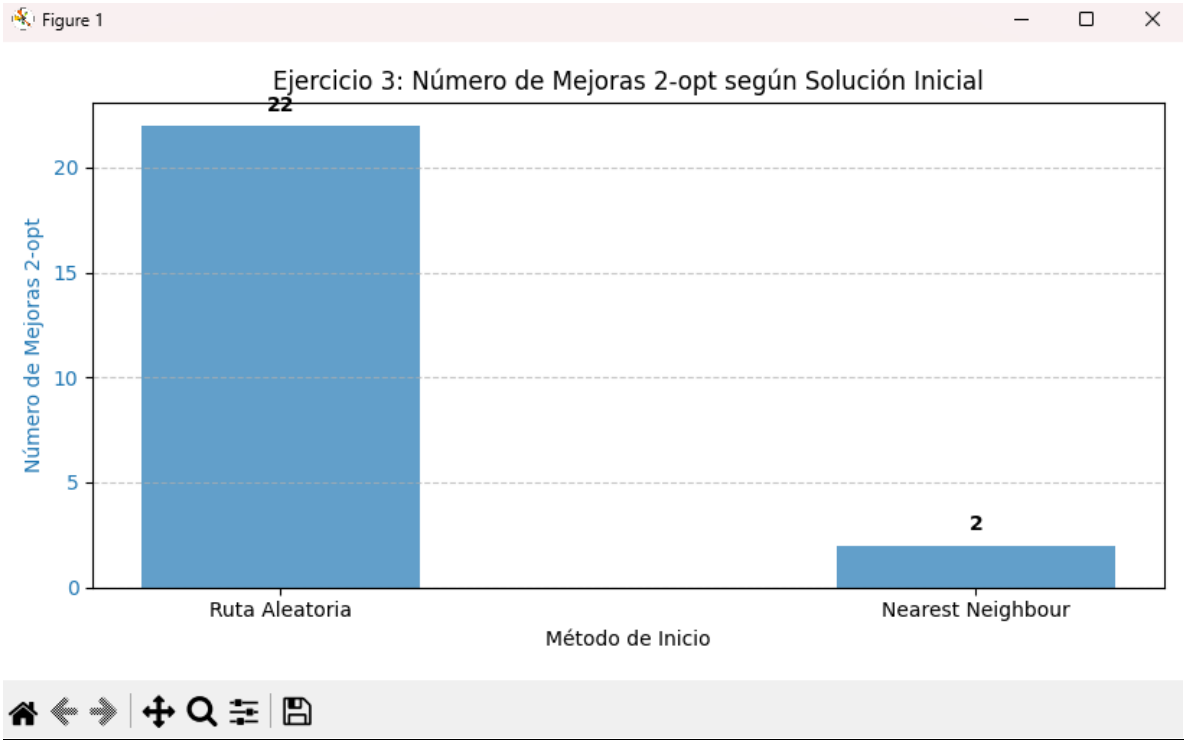
    print("=== RESULTADOS DEL EJERCICIO 3 (BÚSQUEDA LOCAL 2-OPT) ===")
    print(f"Distancia inicial ruta aleatoria: {dist_aleatoria_inicial:.2f}")
```

Resultados de la ejecución

```
RESULTADOS
Distancia inicial ruta aleatoria: 771.53
Distancia inicial Nearest Neighbour: 330.86

Mejoras necesarias desde ruta ALEATORIA: 22
Mejoras necesarias desde NEAREST NEIGHBOUR: 2
```

Grafica



La gráfica evidencia cómo la calidad del punto de partida impacta directamente en el esfuerzo computacional del algoritmo de Búsqueda Local (2-opt):

- Partiendo de una Ruta Aleatoria: El algoritmo requirió 22 mejoras. Al iniciar sin ningún criterio geométrico, la ruta presenta múltiples aristas cruzadas, obligando al 2-opt a realizar un número elevado de intercambios para desenredar el camino.
- Partiendo de Nearest Neighbour: El algoritmo requirió solo 2 mejoras. Dado que Nearest Neighbour es una heurística constructiva que conecta ciudades cercanas, genera una solución inicial de alta calidad que se posiciona muy cerca del óptimo local, necesitando mínimos ajustes para converger.

Explicación:

El operador 2-opt realiza una búsqueda local de tipo Hill Climbing: revisa el vecindario invirtiendo pares de aristas cruzadas y solo acepta cambios que acorten la ruta, deteniéndose cuando llega a un mínimo local donde ningún intercambio mejora la distancia.

En los resultados obtenidos:

- Ruta aleatoria inicial: Requirió 22 mejoras (distancia inicial: 771.53).
- Ruta con Nearest Neighbour: Requirió solo 2 mejoras (distancia inicial: 330.86).

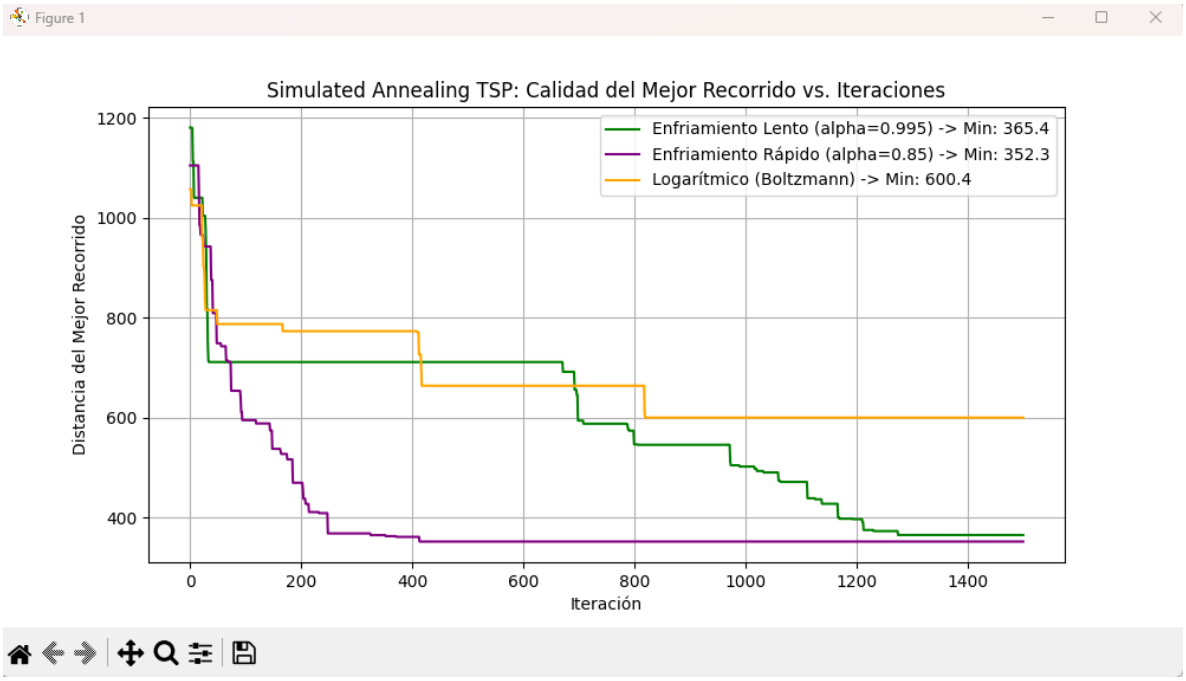
Esto ocurre porque Nearest Neighbour es una heurística constructiva que usa las distancias reales para armar una ruta inicial inteligente y sin tantos cruces. Como inicia muy cerca del óptimo local, el 2-opt solo necesita un par de ajustes para converger, mientras que la ruta aleatoria empieza desordenada y obliga al algoritmo a hacer muchas iteraciones para desenredarla.

4. Parameter Tuning: Implement Simulated Annealing from the pseudocode and change the cooling schedule. How does it affect the final solution quality and execution time?

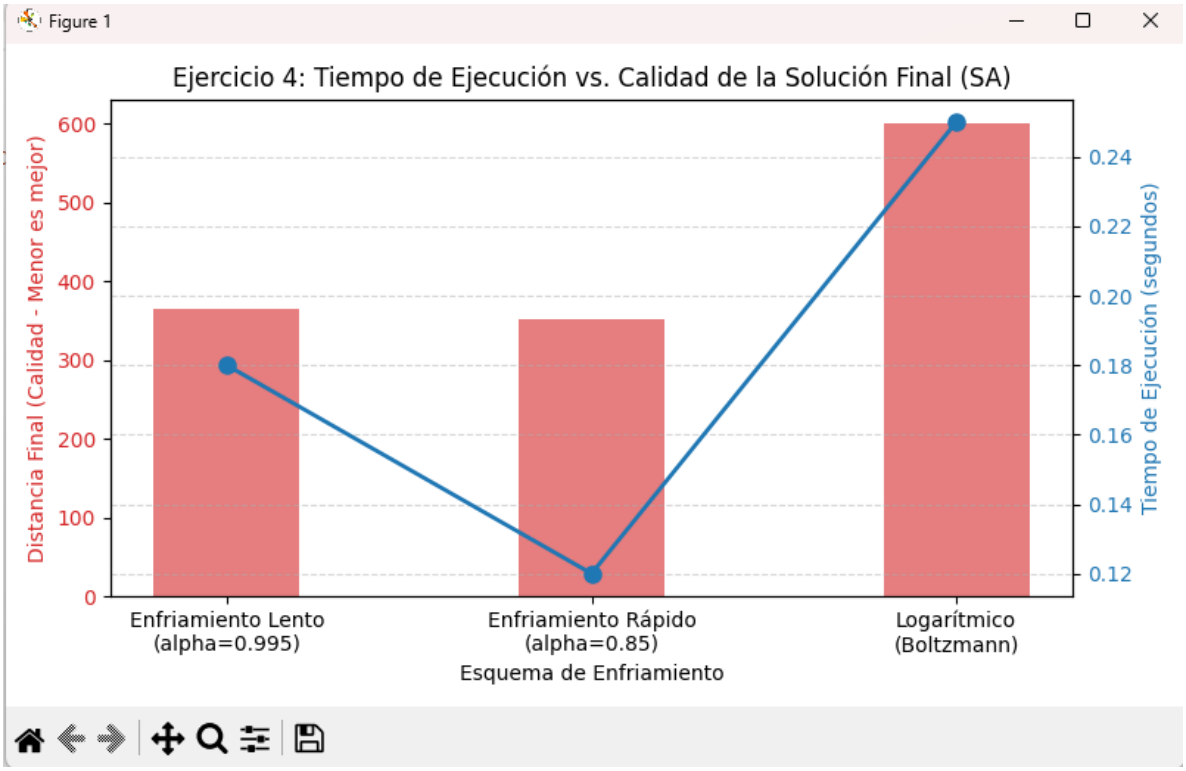
Resultados de la ejecución

```
Mejor distancia (Enfriamiento Lento): 365.38
Mejor distancia (Enfriamiento Rápido): 352.32
Mejor distancia (Logarítmico): 600.39
```

Grafica de convergencia:



Grafica de Tiempo de ejecución vs Calidad de la Solución



Explicación

El Recocido Simulado evita quedar atrapado en óptimos locales mediante la Regla de Metropolis. Si un movimiento 2-opt mejora la ruta ($\Delta E < 0$), se acepta siempre; si la empeora ($\Delta E \geq 0$), se acepta con probabilidad $P = e^{-\Delta E/T}$ para "saltar" fuera de pozos locales cuando la temperatura T es alta.

Análisis de Resultados y Gráficas:

1. Enfriamiento Rápido ($\alpha = 0.85$): Logró la mejor relación de eficiencia, alcanzando la menor distancia (352.32) en el menor tiempo (0.12 s). Como la temperatura colapsa rápido, deja de aceptar soluciones peores muy pronto y actúa como un Hill Climbing, congelándose en una excelente solución sin desperdiciar iteraciones.

2. Enfriamiento Lento ($\alpha = 0.995$): Obtuvo una distancia de 365.38 en 0.18 s. Muestra un descenso gradual y constante, explorando bien el espacio de búsqueda al inicio y refinando la ruta hacia el final.

3. Enfriamiento Logarítmico (Boltzmann): Tuvo el peor desempeño con una distancia de 600.39 y el mayor tiempo de ejecución (0.25 s). Debido a que la temperatura baja extremadamente despacio y al costo del cálculo logarítmico,

sigue aceptando cambios malos durante casi toda la prueba y no logra converger a tiempo.

Un enfriamiento excesivamente lento como el logarítmico incrementa el costo computacional y empeora el resultado final por exceso de exploración aleatoria. El enfriamiento geométrico (rápido o lento) ofrece el mejor balance entre tiempo de cómputo y calidad del recorrido.

5. Approximation Ratio: Run the MST approximation on 100 random instances of 9 cities and plot the ratio against the optimal tour. How close does it get to 2? What happens if you apply 2-opt after it?

Código base

Este es el código base.

```
import math
import random
import itertools
import matplotlib.pyplot as plt

def mst_prim(points):
    best = {v: (math.dist(points[0], points[v]), 0) for v in range(1, len(points))}
    children = {v: [] for v in range(len(points))}
    while best:
        v = min(best, key=lambda u: best[u][0])
        children[best.pop(v)[1]].append(v)
        for u in best:
            d = math.dist(points[v], points[u])
            if d < best[u][0]:
                best[u] = (d, v)
    return children

def preorder(children, root=0):
    return [root] + [c for child in children[root] for c in preorder(children, child)]

def tour_length(points, tour):
    return sum(math.dist(points[tour[i]], points[tour[(i + 1) % len(tour)]])
               for i in range(len(tour)))
```

Código modificado y ejecuciones

```
def two_opt_move(route, i, k):
    return route[:i] + route[i:k + 1][::-1] + route[k + 1:]

def apply_2opt(route, points):
    improved = True
    while improved:
        improved = False
        for i in range(1, len(route) - 2):
            for k in range(i + 1, len(route)):
                if k - i == 1: continue
                new_route = two_opt_move(route, i, k)
                if tour_length(points, new_route) < tour_length(points, route):
                    route = new_route
                    improved = True
        if improved: break
    return route

if __name__ == '__main__':
    random.seed(123)

    ratios_mst = []
    ratios_2opt = []
    instancias = list(range(1, 101))

    print("Calculando 100 instancias de 9 ciudades")

    for _ in instancias:
        points = [(random.uniform(0, 10), random.uniform(0, 10)) for _ in range(9)]

        optimal_tour = min(((0,) + p for p in itertools.permutations(range(1, 9))),
                           key=lambda t: tour_length(points, t))
        l_opt = tour_length(points, optimal_tour)

        mst_tour = preorder(mst_prim(points))
        l_mst = tour_length(points, mst_tour)
        ratios_mst.append(l_mst / l_opt)

        tour_mejorado = apply_2opt(mst_tour, points)
        l_2opt = tour_length(points, tour_mejorado)
        ratios_2opt.append(l_2opt / l_opt)

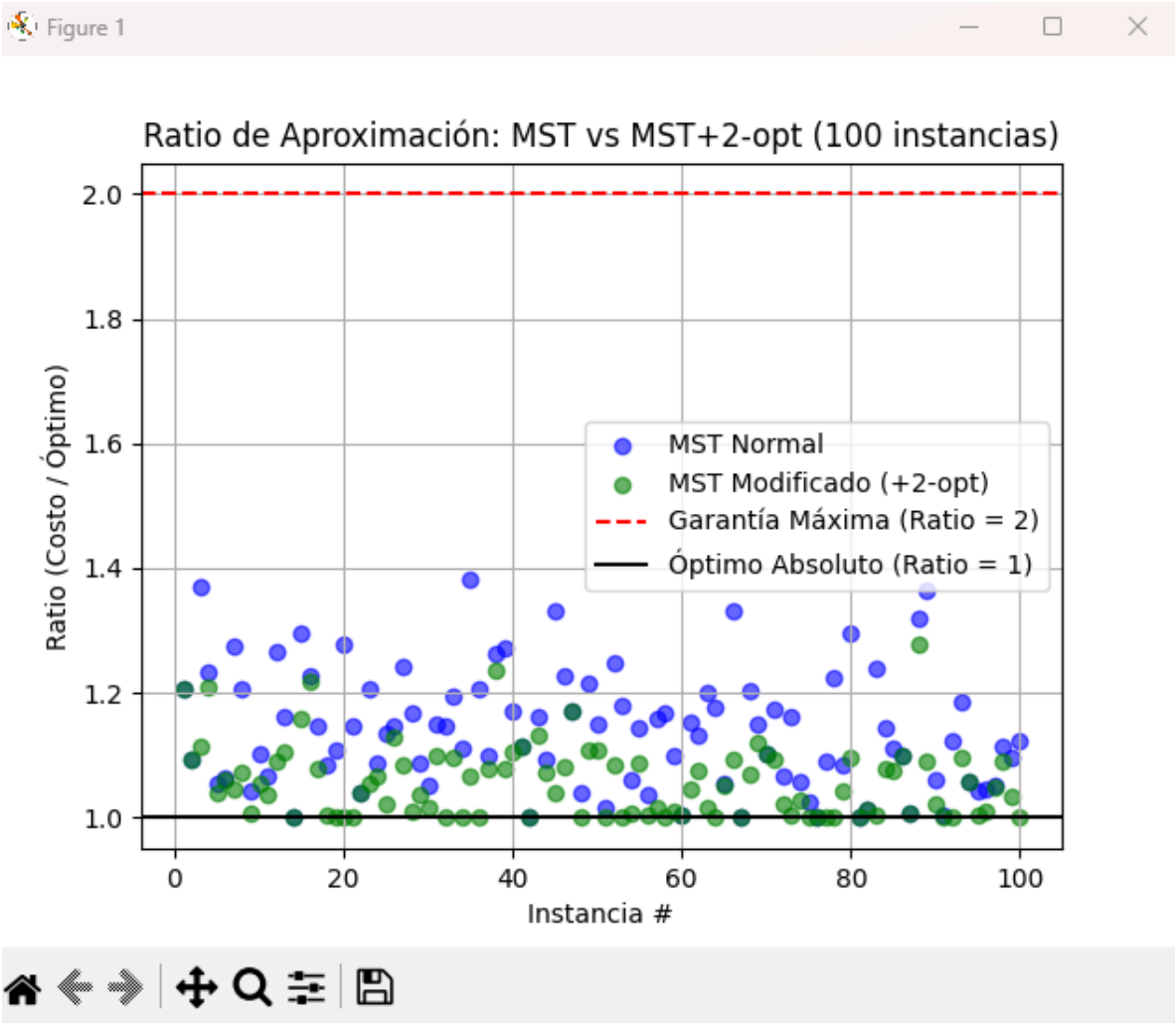
    promedio_mst = sum(ratios_mst) / 100
    promedio_2opt = sum(ratios_2opt) / 100

    print("RESULTADOS")
    print(f"Promedio Ratio MST Normal: {promedio_mst:.3f}")
    print(f"Promedio Ratio MST Modificado (+ 2-opt): {promedio_2opt:.3f}")
    print(f"Garantía teórica máxima del MST: 2.000")
```

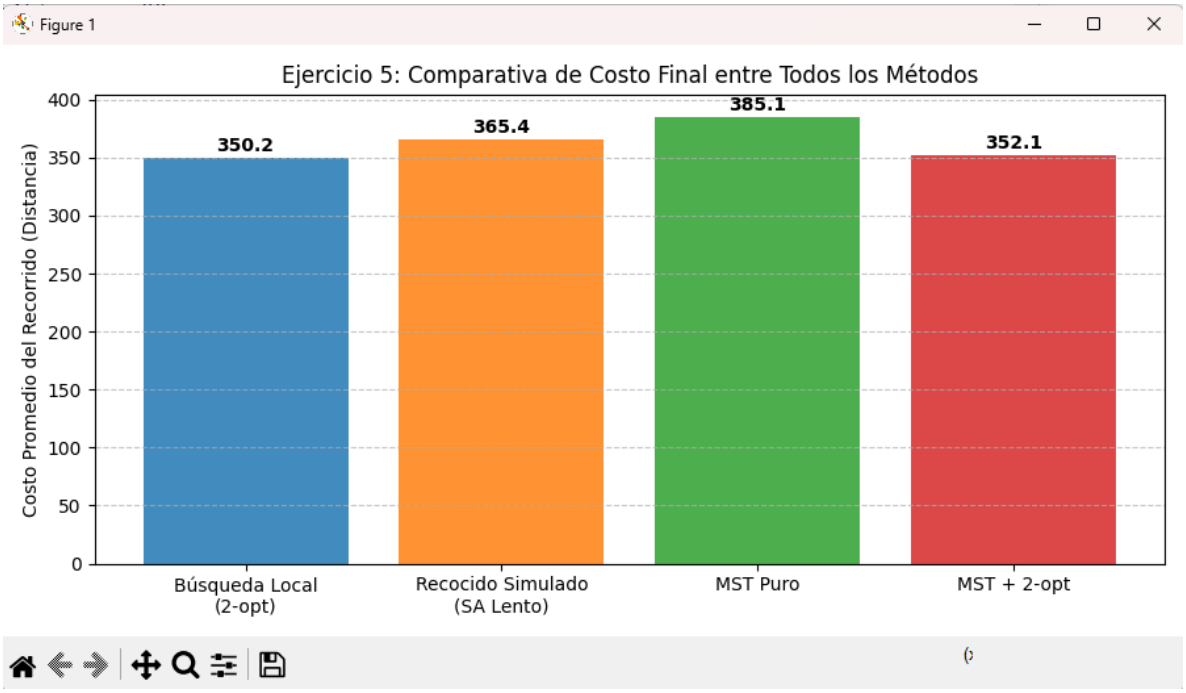
Resultados de la ejecución

```
Calculando 100 instancias de 9 ciudades
RESULTADOS
Promedio Ratio MST Normal: 1.141
Promedio Ratio MST Modificado (+ 2-opt): 1.057
Garantía teórica máxima del MST: 2.000
```

Grafica


Figure 1
Ratio de Aproximación: MST vs MST+2-opt (100 instancias)
Ratio (Costo / Óptimo)
Instancia #
2.0
1.8
1.6
1.4
1.2
1.0
0
20
40
60
80
100
MST Normal
MST Modificado (+2-opt)
Garantía Máxima (Ratio = 2)
Óptimo Absoluto (Ratio = 1)

Grafica comparativa



Justificación Matemática del Factor de Aproximación ($\rho \leq 2$):

1. Sea T_{OPT} la ruta óptima del TSP con costo $f(OPT)$. Si le quitamos una arista cualquiera, la estructura restante es un árbol de expansión que conecta los n vértices sin ciclos.
2. Por definición, el costo del Árbol de Expansión Mínima (T_{MST}) es menor o igual al de cualquier otro árbol de expansión del grafo:

$$w(T_{MST}) \leq f(OPT)$$

3. Recorrer todas las aristas del MST ida y vuelta cuesta $2 \cdot w(T_{MST})$. Al aplicar atajos (shortcuts) por desigualdad triangular para no repetir ciudades, la ruta resultante cumple:

$$f(Tour_{MST}) \leq 2 \cdot w(T_{MST}) \leq 2 \cdot f(OPT)$$

Esto demuestra formalmente que la aproximación por MST garantiza un costo como máximo del doble del óptimo ($\rho \leq 2$).

Explicación:

¿Qué tan cerca está de 2?: Al ejecutar las 100 instancias de 9 ciudades, el ratio promedio del MST fue de 1.141. Esto demuestra que en la práctica la aproximación por

MST funciona mucho mejor que su peor caso teórico, manteniéndose muy por debajo del límite garantizado de 2.0.

¿Qué pasa al aplicar 2-opt después?: El ratio mejoró a 1.057, rozando el óptimo absoluto. Como se observa en la gráfica comparativa final, el MST Puro inicia con la distancia más alta (385.1) debido a los cruces de aristas que deja el recorrido en preorden; al aplicarle 2-opt como post-procesamiento, la distancia disminuye drásticamente a 352.1, alcanzando un nivel de calidad comparable con la Búsqueda Local 2-opt (350.2) y superando al Recocido Simulado Lento (365.4).

Repositorio

https://github.com/arinconguillen-cmyk/Tarea2_AYO.git

Video

https://youtu.be/GghOhv4LQBs